



DPUArduinoSerial

1. Inputs and Outputs:

<dynamically defined>

Inputs and outputs are created according to the packet definition (see below).

2. Static parameters:

Port

Selects a serial port.

Baudrate

Sets the baud rate for the communication (should be 19200). Default: 19200.

NoError

If set to 1 (=true), loading of the simulation will continue even if the Arduino cannot be found, the serial port cannot be accessed or another communication problem exists. An error message will still be shown to indicate something went wrong. Default: 0.

3. Functional principle:

DPUArduinoSerial represents a serial interface to an Arduino board. Communication to the board is realized using the Serial object, which in turn accesses the USB port of the Arduino board. The content of the data packets exchanged between the computer and the Arduino is defined in the following form:

```
<PacketDefinition> =  
(  
    (,)(<VariableName> , <Datatype> [,<DefaultValue> , <Multiplier> ,  
    <Offset>] )  
);
```

The variable names declared in the packet definition are provided as inputs (*SendDefinition*) respectively outputs (*ReceiveDefinition*) of the DPUSocket. The supported data types are listed in the following table:

Data type	Data length	Min	Max
byte	1	-128	+127
ubyte	1	0	+255
word	2	-32768	+32767
uword	2	0	+65535
long	4	-2147483648	+2147483647
ulong	4	0	+4294967295



4. Example code for the Arduino board:

On the Arduino board, the communication is implemented using the object *SILABSerial*. The object *SILABSerial*, in turn, uses the objects *Serial* or *Serial1-3* (depending on the Arduino model) to communicate with SILAB. To not disturb this communication, the selected interface must not be used otherwise in the Arduino project.

The method *Open(Port,Baud)* of object *SILABSerial* allows to select which inter-face and which baud rate are used to communicate with SILAB.

For all Arduino-compatible boards, *Port* = 0 selects the USB interface (which corresponds to the *Serial* object). On Arduino Leonardo boards, *Port* = 1 can be used alternatively to access the *Serial1* object. Arduino Mega boards can use the values *Port* = 1...3 to access the objects *Serial1..Serial3* for the communication.

cSILABSerial is the class that is used to communicate with SILAB. This class is also used by *DPUArduinoSerial* (which runs on the connected computer), so that future changes to the class will affect both ends of the communication and they will automatically remain compatible.

The global instance *SILABSerial* is defined in the include file *SILABSerial.h*. To receive data, the function *serialEvent()* must be implemented.

5. Example:

The following example can be used to control an Arduino Relay Shield (V3) from SILAB.
SILAB DPU configuration:

```
DPUArduinoSerial ~Arduino
{
    Computer = {INO};
    Index = 1;
    Port = 1;
    Baudrate = 19200;
    SendDefinition =
    (
        (Relay1,ubyte),
        (Relay2,ubyte),
        (Relay3,ubyte),
        (Relay4,ubyte)
    );
};
```

The Arduino sketch file:

```
// include SILABSerial for communication
#include <SILABSerial.h>

// define used pins
#define Relay1 4
```



SILAB DRIVING SIMULATION

version Version 7.1 documentation

```
#define Relay2 7
#define Relay3 8
#define Relay4 12

// initialize the read variables with 0
unsigned char uc_Relay1=0;
unsigned char uc_Relay2=0;
unsigned char uc_Relay3=0;
unsigned char uc_Relay4=0;

void setup() {
    // put your setup code here, to run once:
    // open communication with SILAB
    SILABSerial.Open(0,19200);
    // set pin mode to output
    pinMode(Relay1,OUTPUT);
    pinMode(Relay2,OUTPUT);
    pinMode(Relay3,OUTPUT);
    pinMode(Relay4,OUTPUT);
}

void loop() {
    // put your main code here, to run repeatedly:
    // read values from SILAB
    SILABSerial.BeginRead();
    SILABSerial.ReadUByte(uc_Relay1);
    SILABSerial.ReadUByte(uc_Relay2);
    SILABSerial.ReadUByte(uc_Relay3);
    SILABSerial.ReadUByte(uc_Relay4);
    SILABSerial.EndRead();

    // write values to pin to enable/disable relay
    if (uc_Relay1>0) digitalWrite(Relay1,HIGH);
    else             digitalWrite(Relay1,LOW);
    if (uc_Relay2>0) digitalWrite(Relay2,HIGH);
    else             digitalWrite(Relay2,LOW);
    if (uc_Relay3>0) digitalWrite(Relay3,HIGH);
    else             digitalWrite(Relay3,LOW);
    if (uc_Relay4>0) digitalWrite(Relay4,HIGH);
    else             digitalWrite(Relay4,LOW);
}
```



6. Listings:

The following files that implement SILABSerial communication are to be included in the Arduino library. They are provided with your SILAB installation. You can find them in the folder SILAB/src/INO/libraries/SILABSerial/. To use them with the Arduino IDE either select SILAB/src/INO as base folder for your sketchbooks or copy the folder SILABSerial to the libraries folder of the Arduino IDE installation (i.e. %APPDATA%/Arduino/libraries/).

File SILABSerialOptions.h

```
#ifndef _SILABSERIALOPTIONS_H_
#define _SILABSERIALOPTIONS_H_

#define SILABSerialBufferSize 80

#endif
```

File SILABSerial.h

```
#ifndef _SILABSERIAL_H_
#define _SILABSERIAL_H_

#include "cSILABSerial.h"
#ifdef WIVW_ENVIRONMENT
extern cSILABSerial SILABSerial;
extern void serialEvent();
#endif

#endif
```

File SILABSerial.cpp

```
#include "Arduino.h"
#include <SILABSerial.h>

cSILABSerial SILABSerial;

void serialEvent()
{
    SILABSerial.serialEvent();
};
```



cSILABSerialBuffer.h

```
#ifndef _SILABSERIALBUFFER_H_
#define _SILABSERIALBUFFER_H_

#include "SILABSerialOptions.h"

class cSILABSerialBuffer
{
public:
    cSILABSerialBuffer();
    bool DataAvailable();
    // Setzen von Datenbytes als einzelne Bytes
    void SerialSetByte(unsigned char b_Data);
    void WriteUByte(unsigned char uc_Data);
    bool ReadUByte(unsigned char &p_Data);
    unsigned char *GetBuffer();
    int GetBufferLength();
    void Reset();

    bool mb_NewDataAvailable;
    int mi_BufferIDX;
    int mi_BufferRec;
    int mi_BufferReceive;
    unsigned char ma_Buffer[SILABSerialBufferSize];
};
#endif
```



File cSILABSerialBuffer.cpp

```
#include "cSILABSerialBuffer.h"

cSILABSerialBuffer::cSILABSerialBuffer()
{
    mb_NewDataAvailable = false;
    mi_BufferIDX = 0;
    mi_BufferRec = 0;
    mi_BufferReceive = 0;
    Reset();
};

bool cSILABSerialBuffer::DataAvailable()
{
    bool b_Return = mb_NewDataAvailable;
    return b_Return;
};

void cSILABSerialBuffer::Reset()
{
    mb_NewDataAvailable = false;
    mi_BufferIDX = 2;
    mi_BufferRec = 0;
    mi_BufferReceive = 0;
    for (int i=0;i<SILABSerialBufferSize;i++)
        ma_Buffer[i] = 0;
    ma_Buffer[0] = 0x80;
};

void cSILABSerialBuffer::WriteUByte(unsigned char uc_Data)
{
    if (mi_BufferIDX<SILABSerialBufferSize)
    {
        ma_Buffer[0] = ma_Buffer[0] | 0x80;
        int i_Data = mi_BufferRec % 7;
        int i_Sign = mi_BufferRec / 7;
        int i_SIDX = (i_Sign*8)+2;
        int i_DIDX = i_SIDX + (i_Data+1);
        ma_Buffer[i_DIDX] = uc_Data & 0x7f;
        ma_Buffer[i_SIDX] = ma_Buffer[i_SIDX] | ( (uc_Data &
0x80)>>(i_Data+1));
        mi_BufferRec += 1;
        mi_BufferIDX = mi_BufferRec+i_Sign+3;
        ma_Buffer[0] = ((mi_BufferIDX % 0x80) | 0x80);
        ma_Buffer[1] = ((mi_BufferIDX / 0x80) & 0x80);
    };
};
```



```
};  
bool cSILABSerialBuffer::ReadUByte(unsigned char &uc_Data)  
{  
    if (DataAvailable())  
    {  
        int i_Data = mi_BufferRec % 7;  
        int i_Sign = mi_BufferRec / 7;  
        int i_SIDX = (i_Sign*8)+2;  
        int i_DIDX = i_SIDX + (i_Data+1);  
        if ((mi_BufferRec+i_Sign+3)<(mi_BufferIDX))  
        {  
            uc_Data = ma_Buffer[i_DIDX];  
            unsigned char uc_Sign = ((0x80)>>(i_Data+1));  
            if ((ma_Buffer[i_SIDX] & uc_Sign) == uc_Sign)  
                uc_Data |= 0x80;  
            mi_BufferRec += 1;  
            return true;  
        }  
    }  
    return false;  
}  
void cSILABSerialBuffer::SerialSetByte(unsigned char b_Data)  
{  
    // Sync-Byte  
    if ((b_Data & 0x80) == 0x80)  
    {  
        mi_BufferIDX = 0;  
        mi_BufferRec = 0;  
        mi_BufferReceive = (b_Data & 0x7F) ;  
        mb_NewDataAvailable = false;  
        ma_Buffer[mi_BufferIDX] = b_Data;  
    };  
    if ((mi_BufferIDX < SILABSerialBufferSize)&&(mi_BufferIDX>=0))  
    {  
        ma_Buffer[mi_BufferIDX] = b_Data;  
        mi_BufferIDX++;  
    };  
    if (mi_BufferIDX == 2)  
    {  
        mi_BufferReceive += (ma_Buffer[1] & 0x7f) * 0x80;  
    };  
    if ((ma_Buffer[0]>0)&&(mi_BufferReceive==mi_BufferIDX))  
    {  
        mb_NewDataAvailable = true;  
    }
```



```
};  
};  
unsigned char *cSILABSerialBuffer::GetBuffer()  
{  
    return ma_Buffer;  
}  
int cSILABSerialBuffer::GetBufferLength()  
{  
    return mi_BufferIDX;  
}
```




File cSILABSerial.h

```
#ifndef _CSILABSERIAL_H_
#define _CSILABSERIAL_H_

#include "SILABSerialOptions.h"
#include "cSILABSerialBuffer.h"

#ifdef WIIW_ENVIRONMENT
    #include "SYS/DEV/DEV.h"
#else
    #include "Arduino.h"
#endif

class cSILABSerial
{
public:
    cSILABSerial();
    bool Open(int i_Port,int i_Baud);
    bool SendOnReceive;

    void BeginRead();
    bool ReadUByte(unsigned char &uc_Data);
    bool ReadByte ( signed char &c_Data);
    bool ReadUWord(unsigned short &us_Data);
    bool ReadWord ( signed short &s_Data);
    bool ReadULong(unsigned long &ul_Data);
    bool ReadLong ( signed long &l_Data);
    void EndRead();

    void BeginWrite();
    void WriteUByte(unsigned char uc_Data);
    void WriteByte ( signed char c_Data);
    void WriteUWord(unsigned short us_Data);
    void WriteWord ( signed short s_Data);
    void WriteULong(unsigned long ul_Data);
    void WriteLong ( signed long l_Data);
    void EndWrite(int i_Max=0);

    bool ReceiveBufferAvailable();

    void Close();
    void SerialSetByte(unsigned char b_Data);
```



SILAB DRIVING SIMULATION

version Version 7.1 documentation

```
    unsigned int GetReceiveCounter();
    unsigned int GetSendCounter();
#ifdef WIVW_ENVIRONMENT
        void SerialSetBytes(unsigned char *p_Data,int i_Length);
        bool ReadUByte(cVar *p_Var,double d_Mul,double d_Ofs);
        bool ReadByte (cVar *p_Var,double d_Mul,double d_Ofs);
        bool ReadUWord(cVar *p_Var,double d_Mul,double d_Ofs);
        bool ReadWord (cVar *p_Var,double d_Mul,double d_Ofs);
        bool ReadULong(cVar *p_Var,double d_Mul,double d_Ofs);
        bool ReadLong (cVar *p_Var,double d_Mul,double d_Ofs);
#else
        HardwareSerial *m_Serial;
        void serialEvent();
#endif
    cSILABSerialBuffer m_BufferSend;
    cSILABSerialBuffer m_BufferWork;
    cSILABSerialBuffer m_BufferReceive;
    int i_ReceiveSize;
    unsigned int mui_SendCounter;
    unsigned int mui_ReceiveCounter;
    void CopyToWorkBuffer();
#ifdef WIVW_ENVIRONMENT
        cSINSerialPort Serial;
#endif
};

#endif
```



File cSILABSerial.cpp

```
#include "cSILABSerial.h"

bool cSILABSerial::Open(int i_Port,int i_Baud)
{
    mui_SendCounter = 0;
    mui_ReceiveCounter = 0;
    #ifdef WIVW_ENVIRONMENT
        return Serial.Open(i_Port,i_Baud);
    #else
        m_Serial = &Serial;
        #if defined(UBRR1H)
            if (i_Port == 1) m_Serial = &Serial1;
        #endif
        #if defined(UBRR2H)
            if (i_Port == 2) m_Serial = &Serial2;
        #endif
        #if defined(UBRR3H)
            if (i_Port == 3) m_Serial = &Serial3;
        #endif
        m_Serial->begin(i_Baud);
        return true;
    #endif
};

void cSILABSerial::Close()
{
    #ifdef WIVW_ENVIRONMENT
        Serial.Close();
    #endif
}

cSILABSerial::cSILABSerial()
{
    i_ReceiveSize = SILABSerialBufferSize;
    SendOnReceive = true;
};

void cSILABSerial::BeginRead()
{
    #ifdef WIVW_ENVIRONMENT
        int i_Read = 0;
        do
```



SILAB DRIVING SIMULATION

version Version 7.1 documentation

```
{
    unsigned char ma_LBuffer[2500];
    i_Read = Serial.Read(ma_LBuffer,2500);
    SerialSetBytes(ma_LBuffer,i_Read);
} while (i_Read>0);
#endif
m_BufferWork.mi_BufferRec=0;
};

void cSILABSerial::EndRead()
{
};

void cSILABSerial::BeginWrite()
{
    m_BufferSend.Reset();
};

#ifndef WIVW_ENVIRONMENT
void cSILABSerial::serialEvent()
{
    while (m_Serial->available())
    {
        byte b_Data = (byte)(m_Serial->read());
        SerialSetByte(b_Data);
        if (m_BufferReceive.DataAvailable())
        {
            CopyToWorkBuffer();
        }
    };
};
#endif

void cSILABSerial::EndWrite(int i_Max)
{
    #ifdef WIVW_ENVIRONMENT
    Serial.Write(m_BufferSend.ma_Buffer,m_BufferSend.mi_BufferIDX);
    #else
    mui_SendCounter++;
    if (mui_SendCounter>i_Max)
    {
        mui_SendCounter=0;
        if ((mui_ReceiveCounter>0)||(!SendOnReceive))
        {
```



```
        m_Serial->write(m_BufferSend.ma_Buffer,m_BufferSend.mi_BufferIDX)
        ;
        mui_ReceiveCounter = 0;
    }
}
delay(1);
#endif
};

void cSILABSerial::SerialSetByte(unsigned char b_Data)
{
    m_BufferReceive.SerialSetByte(b_Data);
}

#ifdef WIVW_ENVIRONMENT
void cSILABSerial::SerialSetBytes(unsigned char *p_Data,int i_Length)
{
    m_BufferReceive.Reset();
    int i_s = 0;
    for (int i=i_s;i<i_Length;i++)
    {
        m_BufferReceive.SerialSetByte(p_Data[i]);
        if (m_BufferReceive.DataAvailable())
        {
            i_ReceiveSize = m_BufferReceive.GetBufferLength();
            CopyToWorkBuffer();
        }
    }
};
#endif

bool cSILABSerial::ReceiveBufferAvailable()
{
    return m_BufferReceive.DataAvailable();
}

void cSILABSerial::WriteUByte(unsigned char uc_Data)
{
    m_BufferSend.WriteUByte(uc_Data);
};

void cSILABSerial::WriteByte(signed char c_Data)
{

```



```
        m_BufferSend.WriteByte((unsigned char)c_Data);
};

void cSILABSerial::WriteUWord(unsigned short us_Data)
{
    unsigned char uc_Byte;
    uc_Byte = us_Data & 0x00FF;
    m_BufferSend.WriteByte(uc_Byte);
    uc_Byte = (us_Data / 256) & 0x00FF;
    m_BufferSend.WriteByte(uc_Byte);
};

void cSILABSerial::WriteWord(signed short s_Data)
{
    unsigned short us_Data = (unsigned short)s_Data;
    WriteUWord(us_Data);
};

void cSILABSerial::WriteULong(unsigned long ul_Data)
{
    unsigned char uc_Byte;
    uc_Byte = ul_Data & 0x000000FF;
    m_BufferSend.WriteByte(uc_Byte);
    uc_Byte = (ul_Data / 256) & 0x000000FF;
    m_BufferSend.WriteByte(uc_Byte);
    uc_Byte = (ul_Data / 65536) & 0x000000FF;
    m_BufferSend.WriteByte(uc_Byte);
    uc_Byte = (ul_Data / 16777216) & 0x000000FF;
    m_BufferSend.WriteByte(uc_Byte);
};

void cSILABSerial::WriteLong(signed long l_Data)
{
    unsigned long ul_Data = (unsigned long)l_Data;
    WriteULong(ul_Data);
};

bool cSILABSerial::ReadUByte(unsigned char &uc_Data)
{
    return m_BufferWork.ReadUByte(uc_Data);
};

bool cSILABSerial::ReadByte(signed char &c_Data)
{

```



```
        unsigned char uc_Data=0;
        if (m_BufferWork.ReadUByte(uc_Data))
        {
            c_Data = (signed char)uc_Data;
            return true;
        }
        return false;
};

bool cSILABSerial::ReadWord(signed short &s_Data)
{
    unsigned short us_Data = 0;
    if (ReadUWord(us_Data))
    {
        s_Data = (signed short)us_Data;
        return true;
    }
    return false;
}

bool cSILABSerial::ReadUWord(unsigned short &us_Data)
{
    bool b_Return = true;
    unsigned char uc_Byte1=0;
    unsigned char uc_Byte2=0;
    b_Return &= m_BufferWork.ReadUByte(uc_Byte1);
    b_Return &= m_BufferWork.ReadUByte(uc_Byte2);
    if (b_Return)
    {
        us_Data = uc_Byte1 + (uc_Byte2 *256);
        return true;
    }
    return b_Return;
};

bool cSILABSerial::ReadLong(signed long &l_Data)
{
    unsigned long ul_Data = 0;
    if (ReadULong(ul_Data))
    {
        l_Data = (signed long)ul_Data;
        return true;
    }
    return false;
}
```



```
}

bool cSILABSerial::ReadULong(unsigned long &ul_Data)
{
    bool b_Return = true;
    unsigned char uc_Byte1=0;
    unsigned char uc_Byte2=0;
    unsigned char uc_Byte3=0;
    unsigned char uc_Byte4=0;
    b_Return &= m_BufferWork.ReadUByte(uc_Byte1);
    b_Return &= m_BufferWork.ReadUByte(uc_Byte2);
    b_Return &= m_BufferWork.ReadUByte(uc_Byte3);
    b_Return &= m_BufferWork.ReadUByte(uc_Byte4);
    if (b_Return)
    {
        ul_Data = uc_Byte1 + (uc_Byte2 *256) + (uc_Byte3 * 65536) +
        (uc_Byte4*16777216);
    }
    return b_Return;
};

#ifdef WIWV_ENVIRONMENT
bool cSILABSerial::ReadUByte(cVar *p_Var,double d_Mul,double d_Ofs)
{
    unsigned char uc_Data = 0;
    if (ReadUByte(uc_Data))
    {
        (*p_Var) = ((double)uc_Data)*d_Mul+d_Ofs;
        return true;
    }
    return false;
};

bool cSILABSerial::ReadByte(cVar *p_Var,double d_Mul,double d_Ofs)
{
    signed char uc_Data = 0;
    if (ReadByte(uc_Data))
    {
        (*p_Var) = ((double)uc_Data)*d_Mul+d_Ofs;
        return true;
    }
    return false;
};
```




```
bool cSILABSerial::ReadUWord(cVar *p_Var,double d_Mul,double d_Ofs)
{
    unsigned short us_Data = 0;
    if (ReadUWord(us_Data))
    {
        (*p_Var) = ((double)us_Data)*d_Mul+d_Ofs;
        return true;
    }
    return false;
}

bool cSILABSerial::ReadWord(cVar *p_Var,double d_Mul,double d_Ofs)
{
    signed short s_Data = 0;
    if (ReadWord(s_Data))
    {
        (*p_Var) = ((double)s_Data)*d_Mul+d_Ofs;
        return true;
    }
    return false;
}

bool cSILABSerial::ReadULong(cVar *p_Var,double d_Mul,double d_Ofs)
{
    unsigned long ul_Data = 0;
    if (ReadULong(ul_Data))
    {
        (*p_Var) = ((double)ul_Data)*d_Mul+d_Ofs;
        return true;
    }
    return false;
}

bool cSILABSerial::ReadLong(cVar *p_Var,double d_Mul,double d_Ofs)
{
    signed long l_Data = 0;
    if (ReadLong(l_Data))
    {
        (*p_Var) = ((double)l_Data)*d_Mul+d_Ofs;
        return true;
    }
    return false;
}
#endif
```



```
void cSILABSerial::CopyToWorkBuffer()
{
    for (int i=0;i<m_BufferReceive.mi_BufferIDX;i++)
    {
        m_BufferWork.SerialSetByte(m_BufferReceive.ma_Buffer[i]);
    }
    m_BufferReceive.Reset();
    mui_ReceiveCounter++;
}

unsigned int cSILABSerial::GetReceiveCounter()
{
    return mui_ReceiveCounter;
}

unsigned int cSILABSerial::GetSendCounter()
{
    return mui_SendCounter;
}
```